Security Review submission pack — Who Sees What

Which file goes where

Form location	Use
Describe Your Solution — the six text fields	`01-describe-your-solution.md` — paste each block into its matching field
Upload Documentation → Solution Architecture and Usage	`02-solution-architecture-and-usage.md` — Title: *Who Sees What — Solution Architecture and Usage*
Upload Documentation → Sample API Callouts	**Nothing to upload.** The field is required only if the solution needs a web app/service, integrates off-platform, or is API-only. None apply. If the form will not let you past it, upload `02-...` again with the title *No API callouts — package makes zero HTTP callouts*.
Upload Documentation → Security Scanner Reports	Your Code Analyzer HTML output. Regenerate immediately before submitting: `npm run scan`
False Positives Documentation	`03-false-positives.md` — Title: *Who Sees What — False Positives and Accepted Findings*. **Do not tick** "I confirm there are no false positives" — you have six documented suppressions.

Before you submit

1. **Fill every `[SQUARE BRACKET]` placeholder.** They cover the submitted version id, publisher legal name, technical contact, dates, and the Code Analyzer version.
2. **Convert to PDF.** Reviewers expect PDF or Word, not Markdown. From VS Code: Markdown PDF extension, or paste into Word and export.
3. **Regenerate the scanner report** against the exact version you are submitting, so the report and the package match.
4. **Decide on the Graph Engine timeouts** (§4a of the false-positives doc). If you raise the timeout and those two entry points scan clean, replace that section with the clean result — it is a much stronger answer than an explanation.
5. **Prepare the test org.** The reviewer needs a login where the tool shows real data. A fresh scratch org with 7 users and no permission sets will look broken. Seed it: several users on different profiles, a few permission sets, at least one permission set group, and some field-level restrictions so the FLS column has something to show. Pre-assign `PermissionCheckerPermissionSet` to the reviewer's user.

Two things to decide, not just document

The silent audit failure. If the `PE_Inspection_Log__c` insert fails, the inspection still succeeds and the failure is written to a debug log the subscriber cannot see. §5 of the architecture document states this honestly. A reviewer may well ask about it, since the audit trail is the control the product's security story rests on. Consider surfacing a warning in the response DTO before submitting.

Platform Cache in subscriber orgs. `PECacheManager` requests the partition using a `'local.'` prefix, which in a subscriber org resolves to the subscriber's own partitions rather than the packaged one. Caching therefore does nothing in an installed package. It fails safe — the no-cache fallback path is correct and tested — so this is a performance matter, not a security one, and nothing in the UI claims caching is active. Worth fixing, but it does not block submission.

Facts used in these documents

Verified against the package source, not assumed:

- 22 Apex classes: 16 implementation + 6 test
- 22 LWC bundles; only `permissionExplorer` is exposed
- 15 sObjects read; 1 custom object written (`PE_Inspection_Log__c`)
- 0 matches for `HttpRequest`, `new Http()`, `callout:`
- All 16 implementation classes `public with sharing`
- `WITH USER_MODE` / `AccessLevel.USER_MODE` on every query
- 101 Apex tests passing, 88% package coverage
- 6 `@SuppressWarnings`, all in `PEAccessQueryService`, none in the gate rulesets
- Namespace `twinforce`, package id `0HoOW00000000ZV0AY`